1. Create & List
   mkdir test1
   cd test1
   ls
2. Multiple Files
   touch a.txt b.txt c.txt
3. Add Content
   echo "Linux Practice" > a.txt
4. View Content
   cat a.txt
5. Copy File
   cp a.txt copy_a.txt
6. Rename File
   mv b.txt new_b.txt
7. Delete File
   rm c.txt

INTERMEDIATE

8. Move File
   mkdir data
   mv a.txt data/
9. Nested Directory
   mkdir -p project/logs
10. List Recursively
    ls -R

PERMISSIONS

11. Check Permissions
    ls -l
12. Change Permission
    chmod +x copy_a.txt
13. Remove Write Permission
    chmod -w copy_a.txt

REDIRECTION & PIPES

14. Overwrite File
    echo "Hello World" > file.txt
15. Append Data
    echo "Second Line" >> file.txt
16. Use Pipe
    grep "Hello" file.txt
17. Combine Commands
    wc -l file.txt

PROCESS & SYSTEM

18. Current User
    whoami
19. System Date
    date
20. Running Processes
    ps aux

SHELL SCRIPTING

21. Simple Script
    #!/bin/bash
    echo "Hello Linux"

22. Username Script
```
#!/bin/bash
echo "User: $(whoami)"
```
23. Date Script
```
#!/bin/bash
echo "Date: $(date)"
```
24. Variables
```
#!/bin/bash
name="Dhara"
echo "My name is $name"
```
25. Conditional
```
#!/bin/bash
if [ -f file.txt ]
then
echo "File exists"
else
echo "File not found"
fi
```
26. Loop
```
#!/bin/bash
for i in {1..5}
do
echo $i
done
```
27. Combine Script
```
#!/bin/bash
echo "Hello $(whoami)"
echo "Today is $(date)"
```

28. Final Challenge
```
touch f1.txt f2.txt f3.txt
echo "File1" > f1.txt
echo "File2" > f2.txt
echo "File3" > f3.txt
mkdir folder
mv f1.txt folder/
cat f2.txt
```

****

Python problems:

1.n=10
print(n**2, n**3)

2.s=input("enter the string:")
print(s[0], s[-1])

3.for i in range(1,21):
  if (i%3==0):
    print(i)

4.a=int(input("enter a firstnum"))
b=int(input("enter a Secondnum"))
c=int(input("enter a thirdnum"))
if(b>a):
  if(b>c):
    print(f"largest is{b}")
  else:
    print(f"{c} is largest")
else:

```
    print(f"{a} is largest")

5.def correct(n):
    if n>100:
        return True
    else:
        return False
print(correct(101))

6.name=input("Enter Name:")
if name=="admin":
    print("Welcome Admin")
else:
    print("Welcome User")

7.l=[1,2,3,4,5]
count=0
for i in l:
    count+=1
print(count)

8.print("multiplication of 2 table")
n=int(input('enter the number:'))
for i in range(1,11):
    print(f"{n}*{i}={n*i}")

9.list1=eval(input("enter the list:"))
count=0
for i in list1:
    if i>10:
        count+=1
print(count)

10.word=input("enter word to repeat:")
for _ in range(1,5):
    print(word)

11.print("Odd list is")
list1=eval(input("enter list:"))
list2=[]
for i in list1:
    if i%2!=0:
        list2.append(i)
print(list2)

12.print("printing Prime number")
n=int(input("enter number to check it is prime:"))
count=0
for i in range(1,8):
    if n%i==0:
        count+=1
if count>2:
    print("it is not prime")
else:
    print("it is Prime number")print("print second largest")

13.list1 = eval(input("enter list: "))
largest = second = 0
```

```
for i in list1:
    if i > largest:
        second = largest
        largest = i
    elif i > second and i != largest:
        second = i
print(f"second largest is {second}")
```

```
14.sentence = input("Enter a sentence: ")
vowels = "aeiouAEIOU"
v_count = 0
c_count = 0
for ch in sentence:
    if ch.isalpha():   # check only letters
        if ch in vowels:
            v_count += 1
        else:
            c_count += 1
print("Vowels:", v_count)
print("Consonants:", c_count)
```

```
15.d = {}
for i in range(1, 6):
    d[i] = i ** 3
print(d)
```

```
16.sentence = input("Enter a sentence: ")
result=""
for ch in sentence:
    if ch==" ":
        result+="_"
    else:
        result+=ch
print(result)
```

```
17. import random
numbers = []
for i in range(5):
    num = random.randint(1, 100)
    numbers.append(num)
print(numbers)
```

```
18.list1=eval(input("enter list"))
num=int(input("Enter number to count: "))
count=0
for i in list1:
    if i==num:
        count+=1
print(count)
```

sample.txt: I love Python
Java is also popular
Python is easy to learn
C language

```
19. with open("sample.txt", "r") as file:
    for line in file:
```

```
    if "Python" in line:
        print(line.strip())
```

20.try:
age = int(input("Enter age: "))
if age < 0:
raise AgeError("Age cannot be negative!")
print("Valid age:", age)
except AgeError as e:
print("Custom Error:", e)
except ValueError:
print("Please enter a valid number!")

21.s1="listen"
s2="silent"
if sorted(s1)==sorted(s2):
  print("Anagrams")
else:
  print("they are not anagrams")

22.list1 = [1, [2,3], [4,5]]
result = []
for i in list1:
  if type(i) == list:
    result.extend(i)
  else:
    result.append(i)
print(result)

23.keys = ["a","b","c"]
values = [1,2,3]
d = {}
for i in range(len(keys)):
  d[keys[i]] = values[i]
print(d)

24.list1 = eval(input("Enter list: "))
duplicates = []
for i in list1:
  if list1.count(i) > 1 and i not in duplicates:
    duplicates.append(i)
print("Duplicates:", duplicates)

25.list1 = eval(input("Enter list: "))
n = len(list1)
for i in range(n):
  for j in range(0, n-i-1):
    if list1[j] > list1[j+1]:
      # swap
      list1[j], list1[j+1] = list1[j+1], list1[j]
print("Sorted list:", list1)

26.sentence = input("Enter a sentence: ")
words = sentence.split()
result = []
for word in words:
  result.append(word[::-1]) # reverse each word
output = " ".join(result)

```
print(output)
```

27.
```
check = lambda x: "Even" if x % 2 == 0 else "Odd"
print(check(4))
print(check(7))
```

28.data.csv:
```
Name,Age,City
divya,22,hyd
mounika,30,bang
saicharitha,28,Delhi
```

```
import csv
with open("data.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        if int(row["Age"]) > 25:
            print(row)
```

29.
```
data = {
    "student1": {"name": "divya", "age": 20},
    "student2": {"name": "monika", "age": 22}
}
for key in data:
    for v in data[key].values():
        print(v)
```

30.
```
list1 = eval(input("Enter list: "))
even = []
odd = []
for i in list1:
    if i % 2 == 0:
        even.append(i)
    else:
        odd.append(i)
print("Even numbers:", even)
print("Odd numbers:", odd)
```

****

Problem1:
```
class Vehicle:
    def __init__(self, brand, speed):
        self.brand = brand
        self.speed = speed
    def move(self):
        return "Vehicle is moving"
class Car(Vehicle):
    def move(self):
        return "Car is moving fast"
# Create object
car = Car("Toyota", 120)
# Calling method
print(car.move())
```
output: Car is moving fast

problem2:
```
class Employee:
```

```
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
    def display_details(self):
        return f"Name: {self.name}, Salary: {self.salary}"
class Manager(Employee):
    def __init__(self, name, salary, department):
        super().__init__(name, salary)
        self.department = department
    def display_details(self):
        return f"Name: {self.name}, Salary: {self.salary}, Department: {self.department}"
# Create object
m = Manager("Divya", 40000, "IT")
# Call method
print(m.display_details())
```

output: Name: Divya, Salary: 40000, Department: IT

problem3:

```
class Shape:
    def area(self):
        return 0
class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def area(self):
        return self.length * self.width
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius * self.radius
r = Rectangle(5, 3)
c = Circle(4)
print(r.area())
print(c.area())
```

output:15 and 50.24

problem4:

```
class BankAccount:
    def __init__(self, account_number, balance):
        self.account_number = account_number
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount
        return f"Deposited: {amount}, New Balance: {self.balance}"
    def withdraw(self, amount):
        if amount > self.balance:
            return "Insufficient balance"
        else:
            self.balance -= amount
            return f"Withdrawn: {amount}, Remaining Balance: {self.balance}"
acc = BankAccount("12345", 1000)
# Call methods
print(acc.deposit(500))
print(acc.withdraw(200))
print(acc.withdraw(2000))   # invalid case
```

output: Deposited: 500, New Balance: 1500

Withdrawn: 200, Remaining Balance: 1300
Insufficient balance

Problem5:

```
class Student:
    def __init__(self):
        self.__marks = 0
    def set_marks(self, marks):
        if 0 <= marks <= 100:
            self.__marks = marks
        else:
            print("Invalid marks")
    def get_marks(self):
        return self.__marks
s = Student()
# using Set & Get
s.set_marks(85)
print(s.get_marks())
s.set_marks(150)  # invalid
```

output: 85
Invalid marks

Problem6:

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
    def __str__(self):
        return f"{self.title} by {self.author}"
b = Book("Python Basics", "Divya")
# Print object
print(b)
```

output: Python Basics by Divya

problem7:

```
class CartItem:
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity
    def __repr__(self):
        return f"CartItem(name={self.name}, price={self.price}, quantity={self.quantity})"
item = CartItem("Laptop", 50000, 2)
print(item)
```

output: CartItem(name=Laptop, price=50000, quantity=2)

problem:8

```
def square_gen(n):
    for i in range(1, n + 1):
        yield i * i
# Used generator
for num in square_gen(5):
    print(num)
```

output: 1
4
9
16
25

Problem:9

```
def even_gen(n):
    for i in range(1, n + 1):
        if i % 2 == 0:
            yield i
for num in even_gen(10):
    print(num)
```

output: 2
4
6
8
10

Problem:10

```
class Reverse:
    def __init__(self, lst):
        self.lst = lst
        self.i = len(lst) - 1
    def __iter__(self):
        return self
    def __next__(self):
        if self.i < 0:
            raise StopIteration
        val = self.lst[self.i]
        self.i -= 1
        return val
l = [1, 2, 3, 4]
r = Reverse(l)
for x in r:
    print(x)
```

output: 4
3
2
1

Problem:11

```
nums = [1, 2, 3, 4]
# with map
result = list(map(lambda x: x**3, nums))
print(result)
```

output: [1, 8, 27, 64]

problem:12

```
names = ["Divya", "Vidya", "Mounika", "Noyitha"]
# filter
result = list(filter(lambda x: len(x) > 5, names))
print(result)
```

output:
['Mounika', 'Noyitha']

Problem:13

```
from functools import reduce
nums = [1, 2, 3, 4]
# reduce
result = reduce(lambda x, y: x * y, nums)
print(result)
```

output:24

problem:14

```
import time
def timer(func):
    def wrapper():
        start = time.time()
        func()
        end = time.time()
        print("Execution time:", end - start)
    return wrapper
@timer
def run_loop():
    for i in range(1000000):
        pass
run_loop()
```

output: Execution time: 0.05

problem15:

```
def admin_only(func):
    def wrapper(user):
        if user == "admin":
            func(user)
        else:
            print("Access Denied")
    return wrapper
@admin_only
def delete_data(user):
    print("Data deleted by", user)
# Test
delete_data("admin")
delete_data("guest")
```

output:Data deleted by admin
Access Denied

problem:16

```
import json
users = [
    {"name": "Divyal", "email": "divya@gmail.com"},
    {"name": "Mouni", "email": "mouni@yahoo.com"},
    {"name": "rekha", "email": "rekha@gmail.com"}
]
# write JSON
with open("users.json", "w") as f:
    json.dump(users, f)
# read JSON
with open("users.json", "r") as f:
    data = json.load(f)
# filter gmail
for user in data:
    if user["email"].endswith("@gmail.com"):
        print(user["email"])
```

output: divya@gmail.com
rekha@gmail.com

problem:17

```
import re
text = "Contact us at test@gmail.com or hello@yahoo.com"
```

```
# regex
emails = re.findall(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]+", text)
print(emails)
output: ['test@gmail.com', 'hello@yahoo.com']

**********
Projects:
Expense Tracker:
Expenses.json
[
  {
    "amount": 6000.0,
    "category": "food",
    "date": "2026-04-11"
  },
  {
    "amount": 2000.0,
    "category": "rent",
    "date": "2026-04-25"
  },
  {
    "amount": 4000.0,
    "category": "bills",
    "date": "2026-04-30"
  },
  {
    "amount": 5000.0,
    "category": "travel",
    "date": "2026-04-30"
  }
]
-----
Expense_tracker.py:
import json
import os
from datetime import datetime

FILE_NAME = "expenses.json"

# Load data
def load_expenses():
  if os.path.exists(FILE_NAME):
    try:
      with open(FILE_NAME, "r") as file:
        return json.load(file)
    except:
      return []
  return []

# Save data
def save_expenses(expenses):
  with open(FILE_NAME, "w") as file:
    json.dump(expenses, file, indent=4)

# Add Expense
def add_expense(expenses):
  valid_categories = ["food", "clothes", "rent", "bills", "travel"]
```

```
    try:
        amount = float(input("Enter amount: "))

        category = input("Enter category (food, clothes, rent, bills, travel): ").lower()
        if category not in valid_categories:
            print("Invalid category")
            return

        date_input = input("Enter date (YYYY-MM-DD): ")

        try:
            datetime.strptime(date_input, "%Y-%m-%d")
        except:
            print("Invalid date format")
            return

        expense = {
            "amount": amount,
            "category": category,
            "date": date_input
        }

        expenses.append(expense)
        save_expenses(expenses)

        print("Expense added successfully\n")

    except:
        print("Invalid input\n")

# View All Expenses
def view_expenses(expenses):
    if not expenses:
        print("No expenses found\n")
        return

    print("\nAll Expenses:")
    for i, exp in enumerate(expenses, start=1):
        print(f"{i}. {int(exp['amount'])} | {exp['category']} | {exp['date']}")
    print()

# Filter by Category
def filter_by_category(expenses):
    category = input("Enter category: ").lower()

    filtered = [e for e in expenses if e["category"] == category]

    if not filtered:
        print("No matching expenses\n")
        return

    print(f"\nExpenses in '{category}':")
    for exp in filtered:
        print(f"{int(exp['amount'])} | {exp['date']}")
    print()

# Monthly Summary
def monthly_summary(expenses):
```

```
    month = input("Enter month (YYYY-MM): ")

    total = 0
    for exp in expenses:
        if exp["date"].startswith(month):
            total += exp["amount"]

    print(f"\nTotal spending for {month}: {int(total)}\n")

# Delete Expense
def delete_expense(expenses):
    if not expenses:
        print("No expenses to delete\n")
        return

    print("\nExpenses:")
    for i, exp in enumerate(expenses, start=1):
        print(f"{i}. {int(exp['amount'])} | {exp['category']} | {exp['date']}")

    try:
        choice = int(input("Enter expense number to delete: "))

        if 1 <= choice <= len(expenses):
            removed = expenses.pop(choice - 1)
            save_expenses(expenses)
            print(f"Deleted: {int(removed['amount'])} | {removed['category']}\n")
        else:
            print("Invalid number\n")
    except:
        print("Invalid input\n")

# Budget Limit
def budget_limit(expenses):
    month = input("Enter month (YYYY-MM): ")
    limit = float(input("Enter budget limit: "))

    total = 0
    for exp in expenses:
        if exp["date"].startswith(month):
            total += exp["amount"]

    print(f"\nTotal spending: {int(total)}")

    if total > limit:
        print(f"Budget exceeded by {int(total - limit)}\n")
    else:
        print(f"Within budget. Remaining: {int(limit - total)}\n")

# Main Menu
def main():
    expenses = load_expenses()

    while True:
        print("====== Smart Expense Tracker ======")
        print("1. Add Expense")
        print("2. View All Expenses")
        print("3. Filter by Category")
        print("4. Monthly Summary")
```

```
        print("5. Delete Expense")
        print("6. Budget Limit Check")
        print("7. Exit")

        choice = input("Choose an option: ").strip()

        if choice == "1":
            add_expense(expenses)
        elif choice == "2":
            view_expenses(expenses)
        elif choice == "3":
            filter_by_category(expenses)
        elif choice == "4":
            monthly_summary(expenses)
        elif choice == "5":
            delete_expense(expenses)
        elif choice == "6":
            budget_limit(expenses)
        elif choice == "7":
            print("Exiting... Data saved")
            break
        else:
            print("Invalid choice\n")

# Run
if __name__ == "__main__":
    main()
```

---

output:

```
====== Smart Expense Tracker ======
1. Add Expense
2. View All Expenses
3. Filter by Category
4. Monthly Summary
 5. Delete Expense
6. Budget Limit Check
7. Exit
Choose an option: 2
All Expenses: 1. 6000 | food | 2026-04-11 2. 2000 | rent | 2026-04-25 3. 4000 | bills | 2026-04-30 4. 5000 | travel | 2026-04-30
**********
```

Student Management System:

StudentManagementSystem.py

```
import json
import os

FILE_NAME = "students.json"

# Student Class
class Student:
    def __init__(self, name, roll, marks):
        self.name = name
        self.roll = roll
        self.marks = marks

    def to_dict(self):
        return {
            "name": self.name,
```

```
            "roll": self.roll,
            "marks": self.marks
        }

# Load data
def load_students():
    if os.path.exists(FILE_NAME):
        try:
            with open(FILE_NAME, "r") as file:
                return json.load(file)
        except:
            return []
    return []

# Save data
def save_students(students):
    with open(FILE_NAME, "w") as file:
        json.dump(students, file, indent=4)

# Add Student
def add_student(students):
    name = input("Enter name: ")
    roll = input("Enter roll number: ")
    marks = float(input("Enter marks: "))

    # check duplicate roll
    for s in students:
        if s["roll"] == roll:
            print("Student with this roll number already exists\n")
            return

    student = Student(name, roll, marks)
    students.append(student.to_dict())
    save_students(students)

    print("Student added successfully\n")

# View Students
def view_students(students):
    if not students:
        print("No students found\n")
        return

    print("\nAll Students:")
    for s in students:
        print(f"{s['roll']} | {s['name']} | {s['marks']}")
    print()

# Search Student
def search_student(students):
    roll = input("Enter roll number to search: ")

    for s in students:
        if s["roll"] == roll:
            print(f"Found: {s['roll']} | {s['name']} | {s['marks']}\n")
            return

    print("Student not found\n")
```

```
# Update Marks
def update_marks(students):
    roll = input("Enter roll number: ")

    for s in students:
        if s["roll"] == roll:
            new_marks = float(input("Enter new marks: "))
            s["marks"] = new_marks
            save_students(students)
            print("Marks updated successfully\n")
            return

    print("Student not found\n")

# Delete Student
def delete_student(students):
    roll = input("Enter roll number to delete: ")

    for i, s in enumerate(students):
        if s["roll"] == roll:
            students.pop(i)
            save_students(students)
            print("Student deleted successfully\n")
            return

    print("Student not found\n")

# Main Menu
def main():
    students = load_students()

    while True:
        print("====== Student Management System ======")
        print("1. Add Student")
        print("2. View Students")
        print("3. Search Student")
        print("4. Update Marks")
        print("5. Delete Student")
        print("6. Exit")

        choice = input("Choose an option: ")

        if choice == "1":
            add_student(students)
        elif choice == "2":
            view_students(students)
        elif choice == "3":
            search_student(students)
        elif choice == "4":
            update_marks(students)
        elif choice == "5":
            delete_student(students)
        elif choice == "6":
            print("Exiting program")
            break
        else:
            print("Invalid choice\n")
```

```
# Run
if __name__ == "__main__":
    main()
**********
students.json:
[
    {
        "name": "Divya",
        "roll": "201",
        "marks": 400.0
    },
    {
        "name": "parnika",
        "roll": "202",
        "marks": 350.0
    },
    {
        "name": "mounika",
        "roll": "203",
        "marks": 420.0
    }
]
 **********
Output:
====== Student Management System ======
1. Add Student
2. View Students
3. Search Student
4. Update Marks
5. Delete Student
6. Exit
Choose an option: 2

All Students:
201 | Divya | 400.0
202 | parnika | 350.0
203 | mounika | 420.0
204 | lokitha | 380.0
**********
```

3. Library Management System

LibraryManagementSystem.py

```
file = "books.txt"

while True:
    print("\n1.Add 2.Issue 3.Return 4.View 5.Exit")
    ch = input("Enter: ")

    if ch == "1":
        name = input("Book: ")
        author = input("Author: ")
        f = open(file, "a")
        f.write(name + "," + author + ",0\n")   # 0 = available
        f.close()

    elif ch == "2":
        name = input("Book: ")
        user = input("User: ")
```

```
        f = open(file, "r")
        data = f.readlines()
        f.close()

        f = open(file, "w")
        for line in data:
            b = line.strip().split(",")
            if b[0] == name and b[2] == "0":
                f.write(b[0] + "," + b[1] + ",1," + user + "\n")  # 1 = issued
                print("Issued")
            else:
                f.write(line)
        f.close()

    elif ch == "3":
        name = input("Book: ")

        f = open(file, "r")
        data = f.readlines()
        f.close()

        f = open(file, "w")
        for line in data:
            b = line.strip().split(",")
            if b[0] == name and b[2] == "1":
                f.write(b[0] + "," + b[1] + ",0\n")
                print("Returned")
            else:
                f.write(line)
        f.close()

    elif ch == "4":
        f = open(file, "r")
        for line in f:
            b = line.strip().split(",")
            if b[2] == "0":
                print(b[0], "-", b[1])
        f.close()

    elif ch == "5":
        break
```

output:

```
1.Add 2.Issue 3.Return 4.View 5.Exit
Enter: 1
Book: Python Basics
Author: Divya

1.Add 2.Issue 3.Return 4.View 5.Exit
Enter: 1
Book: Core Java
Author: monika

1.Add 2.Issue 3.Return 4.View 5.Exit
Enter: 1
Book: comedy
Author: Mahe
```

1.Add 2.Issue 3.Return 4.View 5.Exit
Enter: 2
Book: comedy
User: sony
Issued

1.Add 2.Issue 3.Return 4.View 5.Exit
Enter: 4
Python Basics - Divya
Core Java – monika
****
4. Log Analyzer Tool
LogAnalyzerTool.py

```
import re

file = "log.txt"

f = open(file, "r")
lines = f.readlines()
f.close()

ips = []
dates = []
errors = []
ip_count = {}

for line in lines:
    # extract IP
    ip = re.findall(r"\d+\.\d+\.\d+\.\d+", line)
    if ip:
        ips.append(ip[0])
        if ip[0] in ip_count:
            ip_count[ip[0]] += 1
        else:
            ip_count[ip[0]] = 1

    # extract date
    date = re.findall(r"\d{4}-\d{2}-\d{2}", line)
    if date:
        dates.append(date[0])

    # find ERROR
    if "ERROR" in line:
        errors.append(line.strip())

# display
print("IPs:", ips)
print("Dates:", dates)

print("\nError Lines:")
for e in errors:
    print(e)

print("\nTotal Errors:", len(errors))

print("\nIP Count:")
for i in ip_count:
```

```
    print(i, "->", ip_count[i])

# save summary
f = open("summary.txt", "w")

f.write("Total Errors: " + str(len(errors)) + "\n")

f.write("\nIP Count:\n")
for i in ip_count:
    f.write(i + " -> " + str(ip_count[i]) + "\n")

f.close()
```

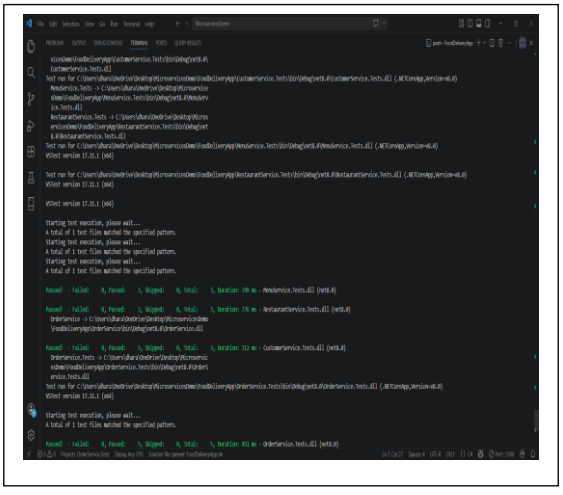
log.txt:
2024-01-01 192.168.1.1 INFO Start
2024-01-02 192.168.1.2 ERROR Failed login
2024-01-03 192.168.1.1 ERROR Crash
2024-01-04 192.168.1.3 INFO Running

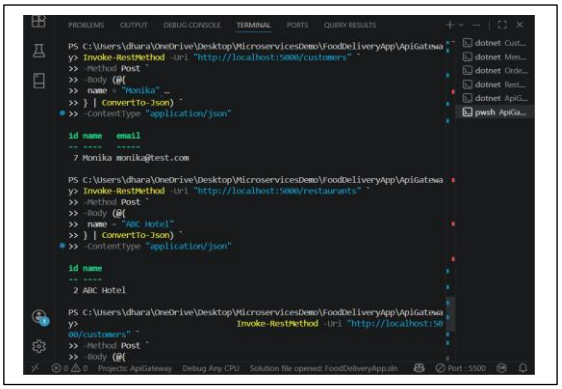
Summary.txt:
Total Errors: 2

IP Count:
192.168.1.1 -> 2
192.168.1.2 -> 1
192.168.1.3 -> 1

Output:
IPs: ['192.168.1.1', '192.168.1.2', '192.168.1.1', '192.168.1.3']
Dates: ['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04']
Error Lines:
2024-01-02 192.168.1.2 ERROR Failed login
2024-01-03 192.168.1.1 ERROR Crash
Total Errors: 2
IP Count:
192.168.1.1 -> 2
192.168.1.2 -> 1
192.168.1.3 -> 1

Food Delivery Platform:




PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
QUERY RESULTS
PS C:\Users\dhara\OneDrive\Desktop\MicroservicesDemo\FoodDeliveryApp\ApiGatewa
y> Invoke-RestMethod -Uri "http://localhost:5000/customers" `
>> -Method Post `
>> -Body (@{
>> name = "Monika" …
>> } | ConvertTo-Json) `
>> -ContentType "application/json"
id name email
-- ---- -----
7 Monika monika@test.com
PS C:\Users\dhara\OneDrive\Desktop\MicroservicesDemo\FoodDeliveryApp\ApiGatewa
y> Invoke-RestMethod -Uri "http://localhost:5000/restaurants" `
>> -Method Post `
>> -Body (@{
>> name = "ABC Hotel"
>> } | ConvertTo-Json) `
>> -ContentType "application/json"
id name
-- ----
2 ABC Hotel
PS C:\Users\dhara\OneDrive\Desktop\MicroservicesDemo\FoodDeliveryApp\ApiGatewa
y> Invoke-RestMethod -Uri "http://localhost:50
00/customers" `
>> -Method Post `
>> -Body (@{
dotnet Cust...
dotnet Men...
dotnet Orde...
dotnet Rest...
dotnet ApiG...
pwsh ApiGa...
Projects: ApiGateway
Debug Any CPU
Solution file opened: FoodDeliveryApp.sln
Port : 5500